CSC 202.2

Computer Programming

Study Notes — Godswill | UNIPORT, Computer Science

Chapter Overview

This chapter covers the object-oriented programming (OOP) foundations that structure how modern software is organized, then moves into the practical tools that keep large codebases manageable — class hierarchy and inheritance, interfaces, packages, namespaces, and modularity — before turning to core data structures (lists, stacks, queues) and the classic algorithms built on top of them (linear and binary search, and the two fundamental graph traversal strategies, BFS and DFS).

1. Object-Oriented Programming Fundamentals

1.1 What Is Object-Oriented Programming?

> Object-Oriented Programming (OOP) is based on the real world, structured around 'objects' — data structures that contain data in the form of fields (attributes) or records. OOP grew out of, and largely outgrew, Procedural Programming, which structures programs primarily as a sequence of procedures/functions acting on data kept separate from them.

In OOP, real-world entities are modeled as Objects, and the functional part of an object — the actions it can perform — is represented by data (attributes) manipulated using functions, commonly called methods (e.g. functions to search, retrieve, edit, or delete that data).

1.2 Classes

A Class is a blueprint for Objects of similar attributes — it defines what data (attributes) and behavior (methods) every object created from it will share. An individual object created from a class is called an instance of that class.

1.3 The Four Pillars of OOP

Pillar	Description
Encapsulation	Binds functions and data together within a class, and typically restricts direct outside access to some of an object's internal details.
Abstraction	Hides complex or unnecessary implementation detail, and only shows the essential features/interface a user of the class actually needs.
Inheritance	Lets a sub/child class reuse (inherit) properties and behavior from a parent class. Can be Single (inheriting from one parent) or Multiple (inheriting from more than one parent).
Polymorphism	Lets objects of different classes be treated through a common interface, with each class able to provide its own specific behavior for shared method names.

2. Class Hierarchy and Program Organization

2.1 Why Class Hierarchy Matters

In OOP, class hierarchy (CH) is essential for organizing code into manageable, modular, and re-usable components. Understanding class hierarchy and its impact on program organization enables developers to write cleaner code, fosters collaboration, and enhances the maintainability of code.

2.2 Fundamentals of Class Hierarchy

> Class Hierarchy is the structure that organizes classes in a tree-like model, where classes can inherit from one another. Classes at the top are called the base/super class, while those inheriting from it are called derived/sub classes. This relationship allows a subclass to inherit attributes (data) and behaviours (functions/methods) from the superclass, thereby promoting code reusability and reducing redundant statements (lines of code).

2.3 Single vs Multiple Inheritance

- Single Inheritance: a subclass inherits attributes and behavior from one parent class.
- Multiple Inheritance: a subclass inherits attributes and behavior from more than one parent class.

Example: consider Person and Student, where Student is the subclass and inherits attributes from the superclass Person. Student can use all the properties and methods of the superclass (Person) and also have its own additional features. Just like a child inherits properties from his/her parents but can still develop their own distinct attitude/behavior, a subclass inherits from its superclass but can still define its own unique behavior on top.

2.4 Single Inheritance in Java — Code Example

```
// Parent class
class Person {
    String name = "John";
    void introduce() {
        System.out.println("my name " + name);
    }
}

// Child class
class Student extends Person {
    void study() {
        System.out.println(name + " is studying");
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.introduce();  // inherited method
        s.study();      // Student's own method
    }
```

```
}
```

3. Benefits of Class Hierarchy

- Enhances Program Organization: gives large codebases a clear, navigable structure.
- Code Reusability: class hierarchy promotes reusability by allowing common properties and methods to be defined once in a base class. Subclasses inherit these features rather than redefining them, reducing code duplication and facilitating easier updates.
- Improved Maintenance: changes to shared behavior can often be made in one place (the superclass) rather than in every subclass individually.
- Enhanced Readability and Organization: a well-structured class hierarchy improves the overall readability of the codebase. This enables developers to quickly understand the relationships and functionalities of different components, making it easier to navigate and collaborate on larger projects.

3.1 Multiple Inheritance Example (Single Class, Multiple Methods)

```
// Main Class
public class Main {
    public static void main(String[] args) {
        Professor prof = new Professor();
        prof.teach();
        prof.research();
        prof.supervise();
    }
}
```

4. Multiple Inheritance via Interfaces in Java

Java doesn't support multiple class inheritance directly, because it leads to ambiguity (confusion) — for example, if two parent classes defined the same method differently, the compiler wouldn't know which version a subclass should inherit. Instead, Java allows a class to implement multiple Interfaces. Java uses the 'Interface' construct to achieve the effect of multiple inheritance safely.

4.1 Interfaces and a Class Implementing Multiple Interfaces — Code Example

```
// Interface 1
interface Teacher {
    void teach();
}

// Interface 2
interface Researcher {
    void research();
}
```

```
// Class implementing both interfaces
class Professor implements Teacher, Researcher {
    @Override
    public void teach() {
        System.out.println("Teaching Students");
    }

    @Override
    public void research() {
        System.out.println("Conducting research");
    }

    public void supervise() {
        System.out.println("Supervising postgraduate students");
    }
}
```

5. Designing an Effective Class Hierarchy

Designing an effective class hierarchy requires careful planning to ensure proper organization and functionality. The following principles can guide a developer in this process:

- Abstraction
- Encapsulation
- Polymorphism

5.1 Common Pitfalls in Class Hierarchy Design

While designing class hierarchy can offer significant advantages, several pitfalls should be avoided:

Deep Inheritance Chains

Excessively deep inheritance chains can complicate the code and make it challenging to trace behaviour. It's advisable to keep the hierarchy as shallow as possible, while still reflecting the real-world relationship among the entities involved (objects with properties and functions/behaviours — a parent class entity and its subclass entities).

Tight Coupling

Classes that depend heavily on one another can become tightly coupled, making the system fragile and difficult to change.

Overusing Inheritance

Not every relationship should be modelled with inheritance. Sometimes, composition can provide better flexibility and organization. It is also important to assess whether inheritance is the most suitable approach for the design at hand.

6. Packages, Namespaces, and Modularity

In advanced programming, the organization and structure of code are paramount. The effective use of packages, namespaces, and modularity doesn't only enhance code readability and maintainability, but also fosters a collaborative environment.

6.1 Packages

> A package is a collection of related classes and interfaces which are grouped together, for ease of use (e.g. an Animal package/class grouping). This concept allows developers to cluster similar functionalities, promoting organization in large codebases.

Benefits of Packages

- Logical Grouping: facilitates logical grouping, which helps in managing and intuitively navigating a codebase.
- Namespace Management: by using packages, developers can easily avoid naming conflicts.
- Access Control: packages can also enforce encapsulation by defining public, private, and protected access modifiers at the package level.

The keyword for declaring packages is 'package'.

```
package com.example.myclass;

private class MyClass {
    // Class implementation
}
```

7. Namespaces

> A namespace is a concept prevalent in programming languages like C++, Python, and JavaScript that provides a scope for identifiers such as variable names to be defined. Similarly to packages, namespaces are essential for managing the names of various entities in large software projects, ensuring that name collisions do not occur.

7.1 Benefits of Namespaces

- Avoiding Conflicts: they prevent naming collisions by allowing developers to define functions, classes, and variables with the same name in different contexts.
- Organizational Clarity: a namespace provides a clear structure to the codebase, making it easier to understand the association between different components.
- Modular Development: namespaces can be leveraged for modular development, so different teams can build features independently without naming clashes.

7.2 Illustrating Variable Scope with a Simple Assignment Example

A simple illustration of how identifiers (like variable names) hold distinct values within a defined scope:

```
Dim A, C As Integer
A = 5
B = 4

C = A     ' C = 5
A = B     ' A = 4
B = C     ' B = 5
```

8. Modularization and Modularity

Modularization is the practice of breaking a program down into separate, self-contained modules — each with its own namespace — thereby reducing interdependencies among classes and components. When approaching modularization, it helps to consider three questions:

- How do we modularize? — deciding how to split code into sensible, cohesive modules.
- Why do we need to modularize? — for benefits like reusability, easier maintenance, and enabling teams to work independently.
- When do we need to modularize? — recognizing the point at which a growing codebase benefits from being split into modules rather than remaining monolithic.

Because each module effectively promotes its own namespace, modularization directly reduces interdependencies among classes, letting components be developed, tested, and reused more independently.

9. Iterators vs Enumerators

Java provides two mechanisms for traversing (moving through) the elements of a collection: the modern Iterator interface, and the older, legacy Enumeration interface.

Feature	Iterator	Enumeration
Introduction	Introduced in the Java 1.2+ Collections Framework	Introduced in the Java legacy 1.0 API
Package	java.util.Iterator	java.util.Enumeration
Applicable Collections	Works with all modern collections, like List, TreeSet, etc.	Works with only legacy classes, like Vector and Hashtable
Traversal Direction	Forward traversal only	Forward traversal only
Element Access Methods	next(), hasNext()	nextElement(), hasMoreElements()
Remove Support	Supports removal via the remove() method	Does not support element removal
Performance	Slightly more efficient	Less efficient, and considered outdated

10. The List Data Structure

10.1 Principle of Operation

A List is an ordered collection of elements, stored in a sequential manner, supporting indexing and access by position.

10.2 Key Attributes of a List

- Ordered: elements are stored in the order they are added.
- Indexed: elements can be accessed by their position, starting from zero.
- Allows Duplicates: a list can contain duplicate elements, and can grow or shrink in size as needed.
- Mutable: elements can be added, removed, or modified after the list is created.

10.3 ArrayList vs LinkedList

Unlike ArrayList, a LinkedList provides dynamic memory allocation and efficient addition and deletion — an ArrayList tends to be slower when adding or removing elements in the middle of the list, since remaining elements must be shifted, whereas a LinkedList can insert/remove by simply relinking neighboring nodes.

11. Stack (A Linear Collection Data Structure)

Stack Principle: Last In, First Out (LIFO) — the most recently added element is the first one removed.

11.1 Basic Stack Operations

- push(): insert an element at/onto the top of the stack.
- pop(): remove the element currently at the top of the stack.

12. Queue (A Linear Collection Data Structure)

Queue Principle: First In, First Out (FIFO) — the earliest added element is the first one removed.

12.1 Basic Queue Operations

- enqueue(): add an element at the rear of the queue.
- dequeue(): remove the element at the front of the queue.
- peek() / front(): view the element at the front of the queue without removing it.

12.2 Applications of Queues

- Managing print jobs, where documents are processed in the order they were submitted.

- Buffering/queuing input in an algorithm that stores incoming items until they can be processed.
- As the core data structure behind Breadth-First Search (see Section 14).

13. Search Algorithms

13.1 Linear Search Algorithm

Linear Search searches a collection in sequential (chronological) order, comparing the target value against each element one at a time until it is found or the collection is exhausted.

- Simple to implement.
- Works on both sorted and unsorted arrays.
- Requires no pre-processing (such as sorting) before it can run.
- Can be slow for large datasets, since it may need to check many (or all) elements before finding a match.

Example: searching the unsorted array [15, 8, 24, 10, 30] for the value 10 — a linear search compares 10 to 15 (no match), then 8 (no match), then 24 (no match), then 10 (match found at index 3).

13.2 Binary Search Algorithm

Binary Search is a fast search technique that works only on sorted data, by repeatedly dividing the search interval in half. It compares the target with the middle element of the current interval: if they're equal, the search ends; if the target is smaller, the search continues in the left half; if the target is larger, the search continues in the right half. This process repeats, narrowing the range each time, until the target is found or the interval becomes empty.

- Requires the array to be sorted beforehand.
- Much faster than linear search on large datasets, since each comparison eliminates roughly half of the remaining elements.

Example: searching the sorted array [9, 14, 18, 23, 27, 31, 45] for the value 23. The middle element is 23 itself in this case, so the search would locate it immediately; for a value like 27, the algorithm would compare 27 to the middle element, determine it lies in the right half, and continue narrowing the search within that half until 27 is found.

13.3 Linear Search vs Binary Search

Aspect	Linear Search	Binary Search
Data requirement	Works on sorted or unsorted data	Requires sorted data
Pre-processing	None required	Array must already be sorted
Speed on large data	Slower — may check every element	Much faster — eliminates half the remaining elements each step
Implementation	Very simple	Slightly more complex

14. Graph Traversal Algorithms: BFS and DFS

14.1 Breadth-First Search (BFS)

BFS explores a graph level by level, starting from a source node, using the Queue principle (FIFO). It guarantees finding the shortest path in an unweighted graph, and visits every reachable node exactly once.

BFS Algorithm Steps

- Start at a source node and add it to the queue, marking it as visited.
- Remove the first node from the queue.
- Visit its unvisited neighbours, mark them as visited, and add them to the queue.
- Repeat until the queue is empty (i.e. all reachable nodes have been visited).

Applications of BFS

- GPS navigation algorithms — finding the shortest route.
- Social network analysis — e.g. finding mutual friends.
- Network broadcasting.
- Solving simple puzzles that require finding the shortest sequence of moves.

14.2 Depth-First Search (DFS)

DFS explores a graph by going as deep as possible along each branch before backtracking, using the Stack principle (LIFO) — either an explicit stack or the function call stack via recursion. Unlike BFS, it does not guarantee the shortest path.

DFS Algorithm Steps

- Start at a source node and mark it as visited.
- Continue moving deeper into unvisited neighbouring nodes.
- When no unvisited neighbour remains at the current node, backtrack to the previous node.
- Repeat until every reachable node has been visited.

Applications of DFS

- Pathfinding — finding a path between two nodes.
- Topological sorting.
- Cycle detection in a graph.
- Solving puzzles (such as mazes) using a backtracking approach.

14.3 BFS vs DFS

Aspect	BFS	DFS
Underlying structure	Queue (FIFO)	Stack (LIFO), or recursion
Exploration style	Explores level by level, outward from the source	Explores as deep as possible before backtracking

Aspect	BFS	DFS
Shortest path (unweighted graph)	Guaranteed	Not guaranteed
Memory usage	Can use more memory on wide graphs (many nodes per level)	Often requires less memory than BFS on wide graphs
Typical uses	GPS navigation, social network analysis, network broadcasting, shortest-sequence puzzles	Pathfinding, topological sorting, cycle detection, maze/backtracking puzzles

14.4 Worked Example: Traversing a Sample Graph

Consider a graph with nodes A, B, C, D, E, F, G, where A connects to B and C, B and C each connect onward to further nodes including D, E, F, and G.

Step	BFS Queue	BFS Visited So Far
1	A	—
2	B, C	A
3	C, D, E	A, B
4	D, E, F, G	A, B, C

A DFS traversal of the same graph would instead push a node's neighbours onto a stack and fully explore one branch (e.g. A → B → D...) before backtracking to explore the next unvisited branch, rather than expanding outward level by level as BFS does.

Chapter Summary

Putting the whole chapter together:

- Object-Oriented Programming organizes code around real-world-inspired Objects and Classes, built on the four pillars: Encapsulation, Abstraction, Inheritance, and Polymorphism.
- Class Hierarchy structures classes in a base/super-class to derived/sub-class tree, enabling code reuse through single or multiple inheritance — with Java using Interfaces to safely achieve multiple-inheritance-like behavior.
- Good class hierarchy design follows abstraction, encapsulation, and polymorphism, while avoiding pitfalls like overly deep inheritance chains, tight coupling, and overusing inheritance where composition would serve better.
- Packages and Namespaces organize and scope code in large projects, preventing naming collisions and enabling modular, collaborative development — which Modularization formalizes by breaking programs into independent, reusable components.
- Java's Iterator (modern) and Enumeration (legacy) provide two ways to traverse collections, with Iterator being the more capable and efficient of the two.
- Lists, Stacks (LIFO), and Queues (FIFO) are foundational linear data structures, each suited to different access patterns.
- Linear Search and Binary Search are the two classic search algorithms, trading Linear Search's simplicity and no pre-processing against Binary Search's much greater speed on sorted, large datasets.
- BFS and DFS are the two fundamental graph traversal strategies — BFS (queue-based, level-by-level,

guarantees shortest path in unweighted graphs) and DFS (stack-based, goes deep before backtracking, better suited to pathfinding, cycle detection, and puzzle-solving).